

Buffer Overflow Vulnerability Analysis

Secure Software Design & Development

JPass Password Manager

Total Vulnerabilities Identified: 31

Members:

Haleema Abid(245053)
Samreen Shoukat(245077)
Hamna Saleem(245047)

April 23,2026

VULN-01 — Unchecked System Property Appended to StringBuilder

File	MenuActionType.java
Overflow Type	Logical Buffer Overflow

Issue Description

`System.getProperties().getProperty("java.version")` and `"java.vendor"` return externally influenced values with no length check. If these properties are manipulated via JVM arguments, an unbounded string gets appended into the `StringBuilder` with no size limit.

Vulnerable Code

```
sb.append("Java version:
").append(System.getProperties().getProperty("java.version")).append("\n");
sb.append(System.getProperties().getProperty("java.vendor"));
```

Corrected Code

Fixed Code

```
String javaVersion = System.getProperties().getProperty("java.version", "Unknown");
String javaVendor  = System.getProperties().getProperty("java.vendor",  "Unknown");
if (javaVersion.length() > 50) javaVersion = javaVersion.substring(0, 50);
if (javaVendor.length() > 100) javaVendor  = javaVendor.substring(0, 100);
sb.append("Java version: ").append(javaVersion).append("\n");
sb.append(javaVendor);
```

Protection and Prevention

Always validate and cap externally sourced strings before appending to any buffer. Use safe defaults and enforce length limits at the point of retrieval.

VULN-02 — Missing Upper-Bound Check on Table Row Index

File	TableListener.java
Overflow Type	Boundary Check Vulnerability

Issue Description

rowAtPoint is only checked against -1 but never validated against table.getRowCount() - 1. If coordinates are spoofed or come from an unexpected source, out-of-bounds row access can occur.

Vulnerable Code

```
int rowAtPoint = table.rowAtPoint(point);
if (rowAtPoint > -1) {
    table.setRowSelectionInterval(rowAtPoint, rowAtPoint);
}
```

Corrected Code

Fixed Code

```
int rowAtPoint = table.rowAtPoint(point);
if (rowAtPoint > -1 && rowAtPoint < table.getRowCount()) {
    table.setRowSelectionInterval(rowAtPoint, rowAtPoint);
}
```

Protection and Prevention

Always enforce both lower and upper bounds when validating array or table indices. Never assume a library method returns a value within safe range.

VULN-03 — Unbounded Clipboard Content Pasted into Text Buffer

File	TextComponentActionType.java
Overflow Type	Logical Buffer Overflow

Issue Description

ClipboardUtils.getClipboardContent() returns clipboard data with no size validation. An attacker can place an extremely large string on the clipboard which gets written directly into the text component's document buffer.

Vulnerable Code

```
component.replaceSelection(ClipboardUtils.getClipboardContent());
```

Corrected Code

Fixed Code

```
String content = ClipboardUtils.getClipboardContent();
if (content != null && content.length() > 10000) {
    content = content.substring(0, 10000);
}
component.replaceSelection(content);
```

Protection and Prevention

Enforce a maximum paste size before writing clipboard data into any component. Validate all externally sourced data at the point of entry.

VULN-04 — Unbounded Exception Message Passed to UI

File	Worker.java
Overflow Type	Logical Buffer Overflow

Issue Description

`e.getMessage()` and `e.getCause().getMessage()` return exception messages of arbitrary length with no bounds checking. A crafted input or malformed file can produce a very large exception message that gets rendered in the UI.

Vulnerable Code

```
if (e.getCause() != null) {
    message = e.getCause().getMessage();
} else {
    message = e.getMessage();
}
AlertDialog.showMessageDialog(this.parent, message);
```

Corrected Code

Fixed Code

```
if (e.getCause() != null) {
    message = e.getCause().getMessage();
} else {
    message = e.getMessage();
}
if (message != null && message.length() > 500) {
    message = message.substring(0, 500);
}
AlertDialog.showMessageDialog(this.parent, message);
```

Protection and Prevention

Truncate all exception messages before displaying them in any UI component. Never expose raw exception text directly to users.

VULN-05 — Unbounded Entry Field Data Written to Table Model

File	EntryDetailsTable.java
Overflow Type	Logical Buffer Overflow

Issue Description

detail.getValue(entry) maps entry fields directly into the table row with no length validation. Fields such as notes and URL have no size cap, allowing arbitrarily large strings to be loaded into the table model buffer.

Vulnerable Code

```
public void addRow(Entry entry) {
    tableModel.addRow(detailsToDisplay.stream()
        .map(detail -> detail.getValue(entry))
        .toArray(Object[]::new));
}
```

Corrected Code

Fixed Code

```
private static String truncate(String value, int maxLength) {
    if (value == null) return "";
    return value.length() > maxLength ? value.substring(0, maxLength) : value;
}

public void addRow(Entry entry) {
    tableModel.addRow(detailsToDisplay.stream()
        .map(detail -> truncate(detail.getValue(entry), 500))
        .toArray(Object[]::new));
}
```

Protection and Prevention

Centralise truncation logic in a reusable helper. Apply length limits at the model layer so no unbounded data ever reaches the UI buffer.

VULN-06 — Unbounded Notes Field and Title Concatenation

File	EntryDialog.java
Overflow Type	Logical Buffer Overflow

Issue Description

entry.getNotes() is set into a JTextArea with no length limit. Additionally, originalTitle is concatenated without bounds checking, causing an unbounded string to be set into the title field buffer.

Vulnerable Code

```
this.notesField.setText(entry.getNotes() == null ? "" : entry.getNotes());  
this.titleField.setText(this.originalTitle + (this.newEntry ? " (copy)" : ""));
```

Corrected Code

Fixed Code

```
String notes = entry.getNotes() == null ? "" : entry.getNotes();  
if (notes.length() > 5000) notes = notes.substring(0, 5000);  
this.notesField.setText(notes);  
  
String safeTitle = this.originalTitle.length() > 200  
    ? this.originalTitle.substring(0, 200) : this.originalTitle;  
this.titleField.setText(safeTitle + (this.newEntry ? " (copy)" : ""));
```

Protection and Prevention

Define maximum field lengths for each entry field type and enforce them consistently before assigning values to UI components.

VULN-07 — Unbounded Custom Symbols and Password Storage

File	GeneratePasswordDialog.java
Overflow Type	Logical Buffer Overflow

Issue Description

customSymbolsField.getText() is appended to the character set string with no length validation. A user can type an arbitrarily large custom symbols string causing unbounded growth. The accepted password is also stored with only an empty check and no upper length bound.

Vulnerable Code

```
String characterSet = "";
for (int i = 0; i < PASSWORD_OPTIONS.length; i++) {
    if (this.checkBoxes[i].isSelected()) {
        characterSet += PASSWORD_OPTIONS[i][1];
    }
}
if (this.customSymbolsCheck.isSelected()) {
    characterSet += this.customSymbolsField.getText();
}
```

Corrected Code

Fixed Code

```
StringBuilder characterSetBuilder = new StringBuilder();
for (int i = 0; i < PASSWORD_OPTIONS.length; i++) {
    if (this.checkBoxes[i].isSelected()) {
        characterSetBuilder.append(PASSWORD_OPTIONS[i][1]);
    }
}
if (this.customSymbolsCheck.isSelected()) {
    String custom = this.customSymbolsField.getText();
    if (custom.length() > 100) custom = custom.substring(0, 100);
    characterSetBuilder.append(custom);
}
String characterSet = characterSetBuilder.toString();
```

Protection and Prevention

Use StringBuilder instead of string concatenation in loops. Always cap user-supplied character sets before appending. Enforce password length limits at the storage point.

VULN-08 — Unbounded Search Criteria and Filename in Title

File	JPassFrame.java
Overflow Type	Logical Buffer Overflow

Issue Description

searchCriteria from user input has no length limit before being used in repeated contains() operations. getModel().getFileName() is concatenated into the window title with no length check, allowing a very long file path to produce an unbounded title string.

Vulnerable Code

```
String searchCriteria = this.searchPanel.getSearchCriteria();
entries.stream()
    .filter(entry -> searchCriteria.isEmpty() ||
        entry.getTitle().toLowerCase().contains(searchCriteria.toLowerCase()))
    .forEach(this.entryDetailsTable::addRow);

setTitle(... + getModel().getFileName() + " - " + PROGRAM_NAME);
```

Corrected Code

Fixed Code

```
String raw = this.searchPanel.getSearchCriteria();
String searchCriteria = raw.length() > 200 ? raw.substring(0, 200) : raw;

String fileName = getModel().getFileName() == null ? "Untitled" :
    getModel().getFileName();
if (fileName.length() > 200) fileName = fileName.substring(0, 200) + "...";
setTitle((getModel().isModified() ? "*" : "") + fileName + " - " + PROGRAM_NAME);
```

Protection and Prevention

Cap search input at the retrieval point. Truncate file path strings before embedding in UI labels or window titles.

VULN-09 — Unbounded HTML Label and File Content in StringBuilder

File	MessageDialog.java
Overflow Type	Logical Buffer Overflow

Issue Description

The message parameter is rendered as an HTML label with no length check. `getResourceAsString()` reads an entire file into a `StringBuilder` line by line with no size limit, allowing a large file to consume unbounded heap memory.

Vulnerable Code

```
messagePanel.add(new JLabel("<html>" + String.valueOf(message)
    .replaceAll("\\n", "<br />") + "</html>"), BorderLayout.CENTER);

while ((line = bufferedReader.readLine()) != null) {
    builder.append(line).append('\n');
}
```

Corrected Code

Fixed Code

```
String messageText = String.valueOf(message);
if (messageText.length() > 2000) messageText = messageText.substring(0, 2000) +
"...";
messagePanel.add(new JLabel("<html>" + messageText.replaceAll("\\n", "<br />") +
"</html>"),
    BorderLayout.CENTER);

while ((line = bufferedReader.readLine()) != null) {
    if (builder.length() + line.length() > 100000) break;
    builder.append(line).append('\n');
}
```

Protection and Prevention

Set a hard cap on message string length before rendering HTML. Apply a total-size guard inside file-reading loops to prevent unbounded buffer growth.

VULN-10 — Unbounded Search Input Returned Without Length Check

File	SearchPanel.java
Overflow Type	Logical Buffer Overflow

Issue Description

getSearchCriteria() returns raw text field content directly with no length restriction. Every caller that uses this value for filtering or display receives an unbounded string.

Vulnerable Code

```
criteria = this.criteriaField.getText();
criteria = criteria == null ? "" : criteria.trim();
```

Corrected Code

Fixed Code

```
criteria = this.criteriaField.getText();
criteria = criteria == null ? "" : criteria.trim();
if (criteria.length() > 200) criteria = criteria.substring(0, 200);
```

Protection and Prevention

Enforce input length limits at the source — the getter — so all callers automatically receive bounded values.

VULN-11 — Unbounded String Set into Status Label

File	StatusPanel.java
Overflow Type	Logical Buffer Overflow

Issue Description

setText() passes strings directly to the JLabel with no length validation. Callers supply formatted strings containing entry counts and filenames; a malformed or very long value produces an unbounded label string.

Vulnerable Code

```
public void setText(final String text) {  
    this.label.setText(text);  
}
```

Corrected Code

Fixed Code

```
public void setText(final String text) {  
    if (text != null && text.length() > 300) {  
        this.label.setText(text.substring(0, 300) + "...");  
    } else {  
        this.label.setText(text);  
    }  
}
```

Protection and Prevention

Apply length validation inside all UI setter methods to prevent unbounded strings from reaching rendered components.

VULN-12 — Unbounded Initial Text Loaded into Component Buffer

File	TextComponentFactory.java
Overflow Type	Logical Buffer Overflow

Issue Description

Both `newTextField(String text)` and `newTextArea(String text)` pass the text parameter directly to the component constructor with no length validation, loading unbounded strings into the component buffer.

Vulnerable Code

```
public static JTextField newTextField(String text) {
    JTextField textField = text == null ? new JTextField() : new JTextField(text);
    ...
}

public static JTextArea newTextArea(String text) {
    JTextArea textArea = text == null ? new JTextArea() : new JTextArea(text);
    ...
}
```

Corrected Code

Fixed Code

```
public static JTextField newTextField(String text) {
    if (text != null && text.length() > 500) text = text.substring(0, 500);
    JTextField textField = text == null ? new JTextField() : new JTextField(text);
    ...
}

public static JTextArea newTextArea(String text) {
    if (text != null && text.length() > 10000) text = text.substring(0, 10000);
    JTextArea textArea = text == null ? new JTextArea() : new JTextArea(text);
    ...
}
```

Protection and Prevention

Factory methods should enforce field-appropriate size limits. Centralising this logic in the factory ensures every created component is protected consistently.

VULN-13 — Unbounded Password Char Array in Key Derivation

File	CryptUtils.java
Overflow Type	Logical Buffer Overflow

Issue Description

The text char array is passed to PBEKeySpec with no length check. An arbitrarily large password causes unbounded data to be processed during key derivation. The exception message is also concatenated without bounds into the RuntimeException.

Vulnerable Code

```
KeySpec spec = new PBEKeySpec(text, salt, iteration, 256);
...
throw new RuntimeException("Could not generate PBKDF2-HMAC-SHA256 key: " +
    e.getMessage());
```

Corrected Code

Fixed Code

```
char[] safeText = text;
if (text != null && text.length > 1024) {
    safeText = new char[1024];
    System.arraycopy(text, 0, safeText, 0, 1024);
}
KeySpec spec = new PBEKeySpec(safeText, salt, iteration, 256);
...
String msg = e.getMessage();
if (msg != null && msg.length() > 200) msg = msg.substring(0, 200);
throw new RuntimeException("Could not generate PBKDF2-HMAC-SHA256 key: " + msg);
```

Protection and Prevention

Enforce a maximum password length before key derivation. Sanitise all exception messages used in string concatenation.

VULN-14 — Unbounded String Loaded into Clipboard Buffer

File	ClipboardUtils.java
Overflow Type	Logical Buffer Overflow

Issue Description

setClipboardContent() passes str directly to StringSelection with no length validation. An arbitrarily large string is loaded entirely into clipboard memory with no upper bound.

Vulnerable Code

```
public static void setClipboardContent(String str) throws Exception {
    ...
    StringSelection selection = new StringSelection(str);
    Toolkit.getDefaultToolkit().getSystemClipboard().setContents(selection,
selection);
}
```

Corrected Code

Fixed Code

```
public static void setClipboardContent(String str) throws Exception {
    if (str == null || str.isEmpty()) { clearClipboardContent(); return; }
    if (str.length() > 100000) str = str.substring(0, 100000);
    StringSelection selection = new StringSelection(str);
    Toolkit.getDefaultToolkit().getSystemClipboard().setContents(selection,
selection);
}
```

Protection and Prevention

Cap clipboard content length before passing to system clipboard APIs. Define a reasonable maximum based on typical use cases.

VULN-15 — Unbounded Property String Split into Array

File	Configuration.java
Overflow Type	Logical Buffer Overflow

Issue Description

getArray() calls prop.split(",") on a raw property value with no length check. A maliciously long or crafted property value causes unbounded array creation.

Vulnerable Code

```
public String[] getArray(String key, String[] defaultValue) {
    String prop = getProperty(key);
    if (prop != null) {
        return prop.split(",");
    }
    return defaultValue;
}
```

Corrected Code

Fixed Code

```
public String[] getArray(String key, String[] defaultValue) {
    String prop = getProperty(key);
    if (prop != null) {
        if (prop.length() > 1000) prop = prop.substring(0, 1000);
        return prop.split(",");
    }
    return defaultValue;
}
```

Protection and Prevention

Validate and truncate all configuration property values before processing. Never pass unbounded external strings directly to split or tokenise operations.

VULN-16 — Unbounded Date String Passed to Parser and Logger

File	DateUtils.java
Overflow Type	Logical Buffer Overflow

Issue Description

dateString is passed directly into LocalDateTime.parse() and String.format() inside the log statement with no length validation. A very long malformed date string causes unbounded data to be processed in parsing and logging.

Vulnerable Code

```
public static String formatIsoDateTime(String dateString, DateTimeFormatter
formatter) {
    LocalDateTime dateTime;
    try {
        dateTime = LocalDateTime.parse(dateString,
DateTimeFormatter.ISO_LOCAL_DATE_TIME);
    } catch (...) {
        LOG.log(Level.WARNING, String.format("Could not parse timestamp [%s]...",
dateString, ...));
    }
}
```

Corrected Code

Fixed Code

```
public static String formatIsoDateTime(String dateString, DateTimeFormatter
formatter) {
    if (dateString != null && dateString.length() > 50) {
        dateString = dateString.substring(0, 50);
    }
    LocalDateTime dateTime;
    try {
        dateTime = LocalDateTime.parse(dateString,
DateTimeFormatter.ISO_LOCAL_DATE_TIME);
    } catch (...) { ... }
}
```

Protection and Prevention

Truncate external date strings before parsing. Always cap values embedded in log format strings to prevent unbounded log output.

VULN-17 — Unbounded Input Iterated into StringBuilder

File	StringUtils.java
Overflow Type	Logical Buffer Overflow

Issue Description

stripNonValidXMLCharacters() processes in character by character in a loop with no input length check. An arbitrarily large input string causes the StringBuilder to grow unbounded, consuming memory proportional to input size.

Vulnerable Code

```
public static String stripNonValidXMLCharacters(final String in) {
    if (in == null || in.isEmpty()) return in;
    StringBuilder out = new StringBuilder();
    for (int i = 0; i < in.length(); i++) {
        ...
        out.append(current);
    }
    return out.toString();
}
```

Corrected Code

Fixed Code

```
public static String stripNonValidXMLCharacters(final String in) {
    if (in == null || in.isEmpty()) return in;
    String safeIn = in.length() > 50000 ? in.substring(0, 50000) : in;
    StringBuilder out = new StringBuilder();
    for (int i = 0; i < safeIn.length(); i++) {
        ...
        out.append(current);
    }
    return out.toString();
}
```

Protection and Prevention

Always validate input length before entering processing loops that produce output proportional to input size.

VULN-18 — Unbounded Entry List Growth with No Size Cap

File	Entries.java
Overflow Type	Logical Buffer Overflow

Issue Description

getEntry() returns the internal list directly with no size limit. Callers can keep adding entries indefinitely. A crafted XML file with thousands of entries causes unbounded list growth in memory.

Vulnerable Code

```
public List<Entry> getEntry() {
    if (entry == null) {
        entry = new ArrayList<>();
    }
    return this.entry;
}
```

Corrected Code

Fixed Code

```
private static final int MAX_ENTRIES = 10000;

public List<Entry> getEntry() {
    if (entry == null) {
        entry = new ArrayList<>();
    }
    if (entry.size() > MAX_ENTRIES) {
        return entry.subList(0, MAX_ENTRIES);
    }
    return this.entry;
}
```

Protection and Prevention

Define a maximum entry count constant and enforce it at list access time. Validate list sizes when loading from external sources such as XML files.

VULN-19 — Unbounded Strings in All Entry Field Setters

File	Entry.java
Overflow Type	Logical Buffer Overflow

Issue Description

All field setters (setTitle, setUrl, setUser, setPassword, setNotes) accept strings of arbitrary length with no validation. Malformed or crafted XML input can inject excessively large values into any field.

Vulnerable Code

```
public void setTitle(String value)    { this.title    = value; }
public void setUrl(String value)     { this.url     = value; }
public void setUser(String value)    { this.user    = value; }
public void setPassword(String value){ this.password = value; }
public void setNotes(String value)   { this.notes   = value; }
```

Corrected Code

Fixed Code

```
public void setTitle(String value) {
    if (value != null && value.length() > 200) value = value.substring(0, 200);
    this.title = value;
}
public void setUrl(String value) {
    if (value != null && value.length() > 500) value = value.substring(0, 500);
    this.url = value;
}
public void setPassword(String value) {
    if (value != null && value.length() > 1000) value = value.substring(0, 1000);
    this.password = value;
}
public void setNotes(String value) {
    if (value != null && value.length() > 5000) value = value.substring(0, 5000);
    this.notes = value;
}
```

Protection and Prevention

Enforce field-specific length limits inside every setter. This provides defence-in-depth at the data model layer regardless of how data enters the system.

VULN-20 — Unbounded Input Stream Deserialized Without Size Limit

File	XmlConverter.java
Overflow Type	Logical Buffer Overflow

Issue Description

mapper.readValue(inputStream, documentClass) reads and deserializes the entire input stream with no size cap. A crafted large XML file causes XmlMapper to consume unbounded memory until OutOfMemoryError.

Vulnerable Code

```
public T read(InputStream inputStream) throws IOException {
    return mapper.readValue(inputStream, documentClass);
}
```

Corrected Code

Fixed Code

```
private static final int MAX_INPUT_SIZE = 10 * 1024 * 1024;

public T read(InputStream inputStream) throws IOException {
    byte[] bytes = inputStream.readNBytes(MAX_INPUT_SIZE + 1);
    if (bytes.length > MAX_INPUT_SIZE) {
        throw new IOException("Input file exceeds maximum allowed size of 10MB.");
    }
    return mapper.readValue(bytes, documentClass);
}
```

Protection and Prevention

Always limit input stream size before deserialization. Reject files exceeding a safe threshold before any parsing begins.

VULN-21 — Unchecked IV Array Length in CBC Constructor

File	Cbc.java
Overflow Type	Heap-Based Buffer Overflow

Issue Description

System.arraycopy(iv, 0, this._current, 0, BLOCK_SIZE) is called with BLOCK_SIZE=16 hardcoded. If iv is shorter than 16 bytes, Java throws ArrayIndexOutOfBoundsException at runtime. There is zero validation of iv.length before the copy.

Vulnerable Code

```
public Cbc(byte[] iv, byte[] key, OutputStream output) {
    this._cipher = new Aes256(key);
    this._current = new byte[BLOCK_SIZE];
    System.arraycopy(iv, 0, this._current, 0, BLOCK_SIZE);
}
```

Corrected Code

Fixed Code

```
public Cbc(byte[] iv, byte[] key, OutputStream output) {
    if (iv == null || iv.length < BLOCK_SIZE)
        throw new IllegalArgumentException("IV must be at least " + BLOCK_SIZE + " bytes.");
    this._cipher = new Aes256(key);
    this._current = new byte[BLOCK_SIZE];
    System.arraycopy(iv, 0, this._current, 0, BLOCK_SIZE);
}
```

Protection and Prevention

Always validate array parameters (null check + length check) at the entry point of every method before any array operation. Create a dedicated validateIV() helper.

VULN-22 — Unchecked Key Array Length in AES-256 Constructor

File	Aes256.java
Overflow Type	Heap-Based Buffer Overflow

Issue Description

KEY_SIZE is hardcoded as 32. If the passed key array is shorter than 32 bytes, System.arraycopy reads beyond the end of the array and throws ArrayIndexOutOfBoundsException. No null or length validation is performed.

Vulnerable Code

```
public Aes256(byte[] key) {
    this._expandedKey = new byte[EXPANDED_KEY_SIZE];
    this._tmp          = new byte[BLOCK_SIZE];
    System.arraycopy(key, 0, this._expandedKey, 0, KEY_SIZE);
}
```

Corrected Code

Fixed Code

```
public Aes256(byte[] key) {
    if (key == null || key.length != KEY_SIZE)
        throw new IllegalArgumentException("AES-256 key must be exactly " +
KEY_SIZE + " bytes.");
    this._expandedKey = new byte[EXPANDED_KEY_SIZE];
    this._tmp          = new byte[BLOCK_SIZE];
    System.arraycopy(key, 0, this._expandedKey, 0, KEY_SIZE);
}
```

Protection and Prevention

AES key size requirements must be enforced at the cryptographic layer boundary. Use a shared key validation utility across all cryptographic classes.

VULN-23 — Unchecked Length Parameter in CBC encrypt() and decrypt()

File	Cbc.java
Overflow Type	Heap-Based Buffer Overflow

Issue Description

The length parameter is accepted with no upper-bound check against data.length. If length > data.length, the loop accesses data[i] beyond the end of the array causing `ArrayIndexOutOfBoundsException`.

Vulnerable Code

```
public void encrypt(byte[] data, int length) throws IOException {
    if (data == null || length <= 0) return;
    for (int i = 0; i < length; ++i) {
        this._overflow[this._overflowUsed++] = data[i];
        ...
    }
}
```

Corrected Code

Fixed Code

```
public void encrypt(byte[] data, int length) throws IOException {
    if (data == null || length <= 0) return;
    if (length > data.length)
        throw new IllegalArgumentException(
            "length (" + length + ") exceeds data.length (" + data.length + ")");
    for (int i = 0; i < length; ++i) {
        this._overflow[this._overflowUsed++] = data[i];
    }
}
```

Protection and Prevention

Never trust caller-supplied length parameters when accessing arrays. Always enforce length <= array.length before entering any array-access loop.

VULN-24 — Unchecked inIndex and outIndex in AES-256 encrypt/decrypt

File	Aes256.java
Overflow Type	Heap-Based Buffer Overflow

Issue Description

Both `System.arraycopy` calls use caller-supplied index parameters with no validation. If `inIndex + BLOCK_SIZE > inBlock.length` or `outIndex + BLOCK_SIZE > outBlock.length`, an `ArrayIndexOutOfBoundsException` is thrown.

Vulnerable Code

```
public void encrypt(byte[] inBlock, int inIndex, byte[] outBlock, int outIndex) {
    System.arraycopy(inBlock, inIndex, this._tmp, 0, BLOCK_SIZE);
    ...
    System.arraycopy(this._tmp, 0, outBlock, outIndex, BLOCK_SIZE);
}
```

Corrected Code

Fixed Code

```
if (inBlock == null || inIndex < 0 || inIndex + BLOCK_SIZE > inBlock.length)
    throw new IllegalArgumentException("Invalid inBlock or inIndex");
if (outBlock == null || outIndex < 0 || outIndex + BLOCK_SIZE > outBlock.length)
    throw new IllegalArgumentException("Invalid outBlock or outIndex");
System.arraycopy(inBlock, inIndex, this._tmp, 0, BLOCK_SIZE);
...
System.arraycopy(this._tmp, 0, outBlock, outIndex, BLOCK_SIZE);
```

Protection and Prevention

When designing methods that accept array indices, document minimum array sizes and enforce them at runtime before any array access.

VULN-25 — Hardcoded IV Size Mismatch in CryptOutputStream

File	CryptOutputStream.java
Overflow Type	Heap-Based Buffer Overflow

Issue Description

The IV size is hardcoded as 16 rather than referencing the BLOCK_SIZE constant in Cbc. If BLOCK_SIZE is ever changed, this constructor produces an IV of the wrong size, causing System.arraycopy in Cbc's constructor to fail.

Vulnerable Code

```
public CryptOutputStream(OutputStream parent, byte[] key) throws IOException {
    byte[] iv = new byte[16];
    Random rnd = CryptUtils.newRandomNumberGenerator();
    rnd.nextBytes(iv);
    parent.write(iv);
    this._cipher = new Cbc(iv, key, parent);
}
```

Corrected Code

Fixed Code

```
public static final int AES_BLOCK_SIZE = 16;

public CryptOutputStream(OutputStream parent, byte[] key) throws IOException {
    byte[] iv = new byte[AES_BLOCK_SIZE];
    Random rnd = CryptUtils.newRandomNumberGenerator();
    rnd.nextBytes(iv);
    parent.write(iv);
    this._cipher = new Cbc(iv, key, parent);
}
```

Protection and Prevention

Never use magic numbers for cryptographic parameter sizes. Define a single source of truth in a shared constants class and reference it everywhere.

VULN-26 — No Key Length Validation in CryptOutputStream

File	CryptOutputStream.java
Overflow Type	Heap-Based Buffer Overflow

Issue Description

Neither constructor validates the key parameter for null or correct length (32 bytes for AES-256). The invalid key propagates through Cbc to Aes256 where System.arraycopy crashes with ArrayIndexOutOfBoundsException.

Vulnerable Code

```
public CryptOutputStream(OutputStream parent, byte[] key, byte[] iv) {
    this._cipher = new Cbc(iv, key, parent);
}

public CryptOutputStream(OutputStream parent, byte[] key) throws IOException {
    ...
    this._cipher = new Cbc(iv, key, parent);
}
```

Corrected Code

Fixed Code

```
private static final int AES_KEY_SIZE = 32;

public CryptOutputStream(OutputStream parent, byte[] key, byte[] iv) {
    if (key == null || key.length != AES_KEY_SIZE)
        throw new IllegalArgumentException("Key must be exactly " + AES_KEY_SIZE +
            " bytes.");
    this._cipher = new Cbc(iv, key, parent);
}
```

Protection and Prevention

Apply the fail-fast principle: validate inputs at the outermost layer rather than letting invalid data propagate through multiple call levels before crashing.

VULN-27 — Unvalidated Salt Length After Generation

File	JPassOutputStream.java
Overflow Type	Heap-Based Buffer Overflow

Issue Description

The code only checks `salt.length > 0` but does not verify `salt.length == expectedSaltLength`. If `generateRandomSalt()` returns a shorter array, the wrong salt is written to disk and passed to the key generator, potentially causing `ArrayIndexOutOfBoundsException` in key derivation.

Vulnerable Code

```
byte[] salt = CryptUtils.generateRandomSalt(fileVersionType.getSaltLength());
if (salt.length > 0) {
    parent.write(salt);
}
this.generatedKey = fileVersionType.getKeyGenerator().apply(key, salt);
```

Corrected Code

Fixed Code

```
int expectedSaltLength = fileVersionType.getSaltLength();
byte[] salt = CryptUtils.generateRandomSalt(expectedSaltLength);
if (salt == null || salt.length != expectedSaltLength) {
    throw new IOException("Salt generation failed: expected " + expectedSaltLength
+ " bytes");
}
parent.write(salt);
this.generatedKey = fileVersionType.getKeyGenerator().apply(key, salt);
```

Protection and Prevention

After every generation or allocation call, assert the returned buffer has the exact expected size. Defensive programming requires the caller to verify output size.

VULN-28 — Multi-byte UTF-8 Emoji Causes File Header Length Confusion

File	JPassStream.java
Overflow Type	Heap-Based Buffer Overflow

Issue Description

The file format identifier includes a Unicode emoji (U+1F512) which encodes to 4 bytes in UTF-8, making the total identifier 9 bytes rather than the expected 5 or 6. Code assuming ASCII length will under-read or over-read bytes causing downstream buffer misalignment.

Vulnerable Code

```
byte[] FILE_FORMAT_IDENTIFIER =
    "JPass\uud83d\uudd12".getBytes(StandardCharsets.UTF_8);

byte[] identifier = readBytes(parent, 5);
```

Corrected Code

Fixed Code

```
byte[] FILE_FORMAT_IDENTIFIER = "JPASS".getBytes(StandardCharsets.US_ASCII);

byte[] identifier = readBytes(parent, FILE_FORMAT_IDENTIFIER.length);
```

Protection and Prevention

File format magic bytes must be pure ASCII or explicit byte sequences. Always use `FILE_FORMAT_IDENTIFIER.length` rather than any hardcoded number.

VULN-29 — Unbounded GZIP Decompression — ZIP Bomb Vulnerability

File	EntriesRepository.java
Overflow Type	Heap-Based Buffer Overflow

Issue Description

GZIPInputStream is opened without any limit on decompressed output size. A crafted GZIP bomb can decompress to gigabytes of data, consuming all available JVM heap memory until OutOfMemoryError is thrown.

Vulnerable Code

```
inputStream = new GZIPInputStream(  
    new CryptInputStream(  
        new JPassInputStream(  
            new BufferedInputStream(  
                new FileInputStream(this.fileName)), this.key)));  
entries = CONVERTER.read(inputStream);
```

Corrected Code

Fixed Code

```
private static final long MAX_DECOMPRESSED_SIZE = 50 * 1024 * 1024;  
  
inputStream = new LimitedInputStream(  
    new GZIPInputStream(new CryptInputStream(...)),  
    MAX_DECOMPRESSED_SIZE);  
entries = CONVERTER.read(inputStream);
```

Protection and Prevention

Always limit decompressed size when processing user-supplied compressed data. Use BoundedInputStream from Apache Commons IO. Validate raw file size before decryption.

VULN-30 — Unsafe Exception Message Concatenation in DocumentProcessException

File	DocumentProcessException.java
Overflow Type	Heap-Based Buffer Overflow

Issue Description

The message parameter is appended to the exception string with no length or content validation. A very large embedded string consumes heap memory. Raw exception messages also expose internal system details to users.

Vulnerable Code

```
public DocumentProcessException(String message) {
    super("Cannot process document due to the following exception:\n"
        + message);
}
```

Corrected Code

Fixed Code

```
private final String safeMessage;

public DocumentProcessException(String internalMessage) {
    super("An error occurred processing the document.");
    this.safeMessage = "An error occurred processing the document.";
}

@Override
public String getMessage() { return safeMessage; }
```

Protection and Prevention

Never concatenate raw exception messages into user-visible strings. Show only predefined safe messages in UI. Write technical details to logs only.

VULN-31 — Padding Oracle Vulnerability in CBC finishDecryption()

File	Cbc.java
Overflow Type	Heap-Based Buffer Overflow

Issue Description

The padding value pad is read from attacker-controlled ciphertext. Although a range check exists, timing differences between valid and invalid padding are detectable. An attacker can recover plaintext one byte at a time without knowing the key via a padding oracle attack.

Vulnerable Code

```
public void finishDecryption() throws DecryptException, IOException {
    int pad = this._outBuffer[BLOCK_SIZE - 1] & 0xff;
    if (pad <= 0 || pad > BLOCK_SIZE) { throw new DecryptException(); }
    int left = BLOCK_SIZE - pad;
    if (left > 0) {
        this._output.write(this._outBuffer, 0, left);
    }
}
```

Corrected Code

Fixed Code

```
public void finishDecryption() throws DecryptException, IOException {
    int pad = this._outBuffer[BLOCK_SIZE - 1] & 0xff;
    boolean valid = (pad >= 1 && pad <= BLOCK_SIZE);
    for (int i = BLOCK_SIZE - pad; i < BLOCK_SIZE; i++) {
        if ((this._outBuffer[i] & 0xff) != pad) valid = false;
    }
    if (!valid) throw new DecryptException();
    int left = BLOCK_SIZE - pad;
    if (left > 0) this._output.write(this._outBuffer, 0, left);
}
```

Protection and Prevention

Use AES-GCM (authenticated encryption) instead of AES-CBC with manual padding. If CBC is required, implement constant-time padding validation and use HMAC-then-encrypt.